

LangChain Mastery Guide — Phase 5: Production

Your primary study reference for Units 13 and 14. Beginner-friendly. Step-by-step. Covers Callbacks, Observability, Streaming, and Batching.

How to Use This Guide

This guide is designed to be read top to bottom. Every chapter builds on the one before it. Do not skip ahead — each concept references the previous one. For each unit, you will find:

- **Concept and Theory** — what it is, why it exists, how it works (with analogies)
- **Practical Lab** — a hands-on coding exercise with step-by-step explanations
- **Knowledge Check** — summary, key takeaways, questions, and exercises

By the end, a **Final Project** combines everything into a production-grade application.

Prerequisites

Before starting Phase 5, make sure you have completed Phases 1–4. You should be comfortable with:

- Creating chains using LCEL (the `pipe()` method)
- Building a basic RAG pipeline (loader, splitter, embeddings, retriever, generator)
- Using `invoke()` to run a chain

You will also need:

- Node.js 18 or higher
- npm or yarn
- An OpenAI API key (stored in a `.env` file)

If you are starting a fresh project for Phase 5 labs:

```
mkdir langchain-phase5 && cd langchain-phase5
npm init -y
npm install @langchain/core @langchain/openai langchain dotenv ts-node
typescript @types/node
npx tsc --init
```

Create your `.env` file:

```
OPENAI_API_KEY=sk-your-key-here
```

Introduction to Phase 5: Why Production Matters

You have learned to build powerful LangChain applications — chains, RAG pipelines, and agents. A prototype that works on your laptop is very different from a system that serves real users reliably. Phase 5 bridges that gap.

Two fundamental concerns separate a prototype from a production system:

1. Observability — Can you see what your application is doing? When something goes wrong, can you find out why? Can you measure cost and performance?

Without observability, you are flying blind. You call `.invoke()`, you get an answer (or an error), and you have no idea what happened in between. For simple chains this is acceptable. For a complex RAG pipeline serving thousands of users, it is not.

2. Performance — Can your system respond fast enough? Can it handle many requests at once? Does it feel responsive to users?

An LLM that takes 8 seconds to respond and shows nothing in the meantime feels broken, even if it is working perfectly. An application that processes 100 documents sequentially instead of concurrently wastes minutes of wall-clock time.

Phase 5 covers the tools and techniques that address both concerns:

- **Unit 13** — Callbacks and LangSmith (observability)
 - **Unit 14** — Streaming and Batching (performance)
-

UNIT 13: Callbacks & Observability

Chapter 1: The LangChain Callback System

1.1 What is a Callback?

The word "callback" sounds technical, but the concept is simple. Imagine you order food at a restaurant. You do not stand at the kitchen window watching every step of cooking. Instead, the restaurant has a system: when your order starts, they put a ticket on the board. When it is ready, they ring a bell. These events happen automatically, and you respond to them.

A callback in LangChain works exactly this way. You define functions that should run when specific things happen — "when the LLM starts, do this", "when the LLM finishes, do that", "when a tool is called, log this". LangChain fires those functions automatically at the right moment.

Why does this exist?

LangChain components — chains, models, retrievers, agents — are designed to be composable and reusable. You should not have to edit the internals of a `ChatOpenAI` class to add logging. The callback system provides a clean way to observe what components do without modifying them.

When do you use callbacks?

- **Debugging** — See every intermediate step, input, and output during development
- **Cost tracking** — Count tokens consumed by every LLM call to monitor spending
- **Latency monitoring** — Measure how long each step takes to identify bottlenecks
- **Logging** — Write structured logs to a file, database, or monitoring service
- **Analytics** — Track user behaviour across your application
- **Real-time feedback** — Update a UI as each token arrives (this overlaps with streaming)

1.2 The Event Lifecycle

LangChain fires events at specific moments during execution. These events form a lifecycle — a predictable sequence of "things that happen" every time you run a chain or agent.

Think of it like a relay race. Each runner (component) picks up the baton (the data), runs their leg, and hands it off. The callback system lets you put a camera at every handoff point.

Here are the core events, grouped by component type:

LLM and Chat Model Events:

Event	When it fires	What data you get
<code>handleLLMStart</code>	The moment before the LLM API is called	The prompt being sent
<code>handleLLMNewToken</code>	Each time a new token arrives (during streaming)	The token string
<code>handleLLMEnd</code>	After the LLM finishes generating	The full output text and token usage
<code>handleLLMError</code>	If the LLM throws any error	The error object

Chain and Runnable Events:

Event	When it fires	What data you get
<code>handleChainStart</code>	Before a chain or runnable begins	The chain's class name, inputs
<code>handleChainEnd</code>	After a chain or runnable finishes	The outputs
<code>handleChainError</code>	If the chain throws	The error object

Tool Events (relevant when using Agents):

Event	When it fires	What data you get
<code>handleToolStart</code>	Before an agent calls a tool	The tool name and input
<code>handleToolEnd</code>	After the tool returns	The tool's output
<code>handleToolError</code>	If the tool throws	The error object

Agent Events:

Event	When it fires	What data you get
handleAgentAction	When the agent decides which tool to use	The chosen action and reasoning
handleAgentEnd	When the agent concludes	The final answer

The nesting principle: Events nest. If your chain contains an LLM, the sequence will be: `handleChainStart` → `handleLLMStart` → `handleLLMEnd` → `handleChainEnd`. The chain wraps the LLM. If your chain contains a retriever which contains an embedding model, the nesting goes even deeper. LangSmith (covered later) visualises this nesting as a tree.

1.3 How to Attach Callbacks

You have two options for attaching callback handlers to your components.

Option A: Constructor-level callbacks

Pass the handler when you create the component. The handler will fire for every call made by that component throughout its lifetime.

```
import { ConsoleCallbackHandler } from "@langchain/core/tracers/console";

const handler = new ConsoleCallbackHandler();

const model = new ChatOpenAI({
  modelName: "gpt-3.5-turbo",
  callbacks: [handler], // fires on every call to this model
});
```

Option B: Runtime callbacks via `RunnableConfig`

Pass the handler at `.invoke()` time. The handler fires only for that specific call, not all future calls.

```
const result = await chain.invoke(
  { question: "what is gravity?" },
  { callbacks: [handler] } // fires only for this invocation
);
```

Which should you use?

- Use **constructor-level** when you always want to observe that component (e.g., a production cost tracker that should never be skipped).
- Use **runtime** when you want to observe only specific calls (e.g., verbose logging only in debug mode, or per-user callbacks that need user context injected at call time).

You can use both at the same time. LangChain will fire both handlers.

1.4 Writing Your Own Callback Handler

LangChain provides a base class called `BaseCallbackHandler`. To write a custom handler, you extend it and override only the event methods you care about. You do not have to implement all

events — only the ones relevant to your use case.

Here is the general pattern:

```
import { BaseCallbackHandler } from "@langchain/core/callbacks/base";
import { LLMResult } from "@langchain/core/outputs";
import { Serialized } from "@langchain/core/load/serializable";

class MyCallbackHandler extends BaseCallbackHandler {
  // Every handler must have a unique name property.
  // This name appears in logs and debugging output.
  name = "MyCallbackHandler";

  // Override event methods you want to respond to.
  // Any method you do not override is simply ignored.

  async handleLLMStart(llm: Serialized, prompts: string[]): Promise<void> {
    // Called when the LLM is about to start.
    // `llm` contains the model class info.
    // `prompts` is the array of prompt strings being sent.
    console.log("LLM is starting...");
    console.log("Prompt:", prompts[0]);
  }

  async handleLLMEnd(output: LLMResult): Promise<void> {
    // Called when the LLM finishes.
    // `output.llmOutput.tokenUsage` contains token counts.
    const usage = output.llmOutput?.tokenUsage;
    console.log("LLM finished. Tokens used:", usage?.totalTokens);
  }

  async handleChainStart(
    chain: Serialized,
    inputs: Record<string, unknown>
  ): Promise<void> {
    console.log("Chain started with inputs:", JSON.stringify(inputs));
  }

  async handleChainEnd(outputs: Record<string, unknown>): Promise<void> {
    console.log("Chain finished with outputs:", JSON.stringify(outputs));
  }

  async handleLLMError(err: Error): Promise<void> {
    console.error("LLM error occurred:", err.message);
  }
}
```

Important rules for custom handlers:

1. All methods are `async` — even if you only do synchronous work inside them.
2. Methods should not throw exceptions — if an error occurs inside a callback, log it and return gracefully. A crashing callback can break your chain.
3. Keep handlers lightweight — do not perform slow operations (database writes, network calls) inside a callback unless you use a queue to offload the work asynchronously. Callbacks add latency to every step.

1.5 The Built-in ConsoleCallbackHandler

LangChain ships with a ready-to-use handler called `ConsoleCallbackHandler`. It prints

every event to the console in a structured, human-readable format. This is the fastest way to see the callback system in action.

```
import { ConsoleCallbackHandler } from "@langchain/core/tracers/console";  
const handler = new ConsoleCallbackHandler();
```

You attach it exactly like any other handler. Every event in your chain will be printed, including the nesting depth, timing, and data. This is ideal for development but too noisy for production.

1.6 Token Usage — Why It Matters and How to Access It

LLM APIs charge per token. Understanding token usage is critical for:

- **Cost management** — Know exactly how much each feature costs
- **Budget alerting** — Send an alert if a user session exceeds a token limit
- **Prompt optimisation** — Identify that your system prompt is 2,000 tokens when it could be 400
- **Anomaly detection** — Alert when a request uses 10x the normal tokens (possible prompt injection)

Token usage is available in the `handleLLMEnd` event via `output.llmOutput?.tokenUsage`. The structure is:

```
{  
  promptTokens: 152,      // tokens in the input (your prompt + context)  
  completionTokens: 87,   // tokens in the output (the model's answer)  
  totalTokens: 239        // sum of the above  
}
```

The split matters. Prompt tokens come from the input you send. Completion tokens come from what the model generates. For RAG, retrieved documents can significantly increase prompt tokens. For agents, the full tool history accumulates in the prompt, so long agent runs can become very expensive.

A simple formula for cost estimation:

```
cost = (promptTokens / 1000 * promptPrice) + (completionTokens / 1000 *  
completionPrice)
```

Where prices depend on the model. You check the current pricing on the provider's website (e.g., platform.openai.com/pricing).

Chapter 2: LangSmith — Professional Observability

2.1 The Problem with Console Logs

In development, `console.log()` statements work fine. But in production:

- Logs disappear when the process restarts
- There is no way to search or filter across thousands of runs

- You cannot correlate what the LLM received vs what it returned
- You cannot see the tree structure of nested chains and sub-chains
- You cannot compare run quality over time
- There is no built-in evaluation system

LangSmith solves all of this. It is a cloud platform built by the LangChain team specifically for observing, debugging, and evaluating LLM applications. Think of it as the purpose-built equivalent of Datadog, but for AI chains instead of web servers.

2.2 Core Concepts in LangSmith

Run — A single execution of any component. One LLM call is a run. One retriever call is a run. One tool use is a run.

Trace — The complete tree of all runs triggered by a single top-level invocation. If you call `chain.invoke(...)`, one trace is created. That trace contains all the nested runs — the retriever run, the embedding run, the LLM run — all connected into a tree.

Think of a trace like a family tree. The top-level chain is the grandparent. Its children are the steps it delegates to. Each of those may have their own children (the retriever calling the vector store, the LLM receiving a formatted prompt). LangSmith draws this tree for you.

Project — A workspace that groups related traces. You might have one project per application (`my-chatbot-prod`), or one per environment (`my-chatbot-staging`), or one per experiment (`rag-chunk-size-test`).

Dataset — A collection of input/output pairs. You can build a dataset from your production traces (real questions your users asked) and use it for testing future versions of your chain.

Evaluation — Running your chain against a dataset and scoring the results automatically. LangSmith can use another LLM as a judge to assess correctness, relevance, and other quality dimensions. This is the backbone of LLM testing.

2.3 How LangSmith Works — Zero Code Changes

This is the remarkable part: to enable LangSmith tracing, you do not change any LangChain code. You only set environment variables. LangChain reads these at startup and automatically routes all run data to LangSmith in the background.

The four required environment variables:

```
LANGCHAIN_TRACING_V2=true
LANGCHAIN_ENDPOINT=https://api.smith.langchain.com
LANGCHAIN_API_KEY=ls__your-api-key-here
LANGCHAIN_PROJECT=your-project-name
```

What each one does:

- `LANGCHAIN_TRACING_V2=true` — Activates the tracing system. Without this, nothing is sent to LangSmith. The V2 refers to the second version of the protocol (more efficient and feature-rich than V1).

- `LANGCHAIN_ENDPOINT` — The URL of the LangSmith server. This is always `https://api.smith.langchain.com` unless you are self-hosting LangSmith on-premise.
- `LANGCHAIN_API_KEY` — Your authentication key. Get this from `smith.langchain.com` → Settings → API Keys. Note the prefix: LangSmith keys start with `ls__` (not `sk-` like OpenAI keys).
- `LANGCHAIN_PROJECT` — The project to file traces under. If the project does not exist, LangSmith creates it automatically.

Critical setup detail: Call `dotenv.config()` before importing any LangChain module.

LangChain reads environment variables when modules are loaded (at import time). If you load the modules first, the env vars are not yet available.

```
// CORRECT order:
import * as dotenv from "dotenv";
dotenv.config(); // must be called first
import { ChatOpenAI } from "@langchain/openai"; // then import LangChain

// WRONG order (LangSmith will NOT be configured):
import { ChatOpenAI } from "@langchain/openai"; // LangChain loads first
import * as dotenv from "dotenv";
dotenv.config(); // too late – LangChain already read env vars
```

2.4 What a Trace Looks Like

Once traces are flowing into LangSmith, navigate to your project at `smith.langchain.com`. You will see a list of traces. Clicking one opens a tree view.

For a RAG pipeline, the trace tree looks something like this:

```
[RunnableSequence]                Total: 2.4s | Total tokens: 521
├── [RunnableParallel]             0.9s
│   ├── [VectorStoreRetriever]    0.8s
│   │   └── [OpenAIEmbeddings]    0.3s | 12 tokens (query embedding)
│   └── [RunnablePassthrough]     <1ms
├── [ChatPromptTemplate]          <1ms
└── [ChatOpenAI]                 1.5s | 509 tokens
    Input: "Context: [retrieved docs...]\nQuestion: [user question]"
    Output: "Based on the provided context..."
```

You can click on any node to see the exact input it received and the exact output it produced. For the LLM node, you see the full formatted prompt — exactly what was sent to the API, not just the template.

This level of detail is invaluable for debugging. Common issues that LangSmith reveals instantly:

- "My retriever is returning irrelevant documents" — visible in the retriever node output
- "My prompt is 4,000 tokens when I thought it was 500" — visible in LLM input token count
- "The agent called the wrong tool" — visible in the tool selection step
- "One chain step is taking 3 seconds out of 4 total" — visible in per-step latency

2.5 Evaluations — Automated Quality Measurement

Once you have a dataset of questions and expected answers, you can run evaluations:

```
Dataset: 50 questions about your company's products
Chain: Your RAG chatbot
Evaluator: LLM-as-judge (checks correctness, relevance, completeness)
```

LangSmith runs your chain against all 50 questions, then uses a second LLM to score each answer. You get a dashboard showing:

- Overall score (e.g., 78% of answers rated "correct")
- Per-question scores (so you know which questions fail)
- Comparison across versions (did the new prompt improve things?)

This is how you test your chain systematically instead of manually reading outputs and guessing.

2.6 Debugging Agents with LangSmith

Agents are uniquely difficult to debug. They are non-deterministic — the same question can produce different tool-call sequences on different runs. Without LangSmith, you might see a wrong final answer but have no idea which step produced incorrect reasoning.

With LangSmith, the full ReAct loop is visible:

```
[AgentExecutor] Total: 4.2s
├── [Step 1] Thought: "I need to search for..."
│   ├── [Tool: web_search] Input: "current population of Paris"
│   │   └── Output: "Paris population 2024: 2.1 million..."
│   └── [Step 2] Thought: "I have the answer now..."
│       └── [ChatOpenAI] Final Answer Output: "The population of Paris is..."
```

You can see precisely what the agent was "thinking", which tool it chose, what input it passed to the tool, and what the tool returned. This makes debugging a 5-minute task instead of a 2-hour one.

Chapter 3: Lab Exercises — Callbacks & Observability

Lab Prerequisites

Make sure your project has the required packages:

```
npm install @langchain/core @langchain/openai langchain dotenv
```

All labs in this unit assume the following `.env` file:

```
OPENAI_API_KEY=sk-your-openai-key
```

LangSmith labs additionally require:

```
LANGCHAIN_TRACING_V2=true
LANGCHAIN_ENDPOINT=https://api.smith.langchain.com
LANGCHAIN_API_KEY=ls__your-langsmith-key
LANGCHAIN_PROJECT=langchain-phase5-study
```

Lab 13.1 — Attach ConsoleCallbackHandler to a Chain

Learning objective: See the callback event lifecycle in action. Understand when each event fires and what data it contains.

Why we are doing this: Before writing custom handlers, you need to see the raw event stream. The `ConsoleCallbackHandler` is the fastest way to make the entire lifecycle visible with no code beyond attaching it.

Create the file: `lab13_1_console_callback.ts`

```
import * as dotenv from "dotenv";
dotenv.config();

import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { ConsoleCallbackHandler } from "@langchain/core/tracers/console";

// Step 1: Instantiate the ConsoleCallbackHandler.
// This is a built-in LangChain handler. It listens to ALL events
// and prints each one to the console with the event name and relevant data.
const consoleHandler = new ConsoleCallbackHandler();

// Step 2: Create a ChatOpenAI model and attach the handler at the constructor
// level.
// This means the handler will fire for EVERY call to this model, not just one.
const model = new ChatOpenAI({
  modelName: "gpt-3.5-turbo",
  temperature: 0.7,
  callbacks: [consoleHandler],
});

// Step 3: Build a simple chain – prompt → model → parser.
// This is the same chain you built in earlier phases.
const prompt = ChatPromptTemplate.fromTemplate(
  "You are a helpful assistant. Answer this question in one sentence: {question}"
);
const parser = new StringOutputParser();
const chain = prompt.pipe(model).pipe(parser);

async function main() {
  console.log("=== Invoking the chain – watch events fire ===\n");

  // Step 4: Invoke the chain normally.
  // You do not need to do anything differently because of the callback.
  // The handler fires automatically at each stage.
  const result = await chain.invoke({
    question: "What is the capital of Japan?",
  });

  console.log("\n=== All events fired. Final result: ===");
  console.log(result);
}

main().catch(console.error);
```

Run it:

```
npx ts-node lab13_1_console_callback.ts
```

What you should see in the console:

You will see structured output from the `ConsoleCallbackHandler` printed before the final result. It will look roughly like:

```
=== Invoking the chain – watch events fire ===

[chain/start] [1:chain:RunnableSequence] Entering Chain run with input:
{ question: 'What is the capital of Japan?' }

[chain/start] [1:2:prompt:ChatPromptTemplate] Entering Chain run with input:
{ question: 'What is the capital of Japan?' }

[chain/end] [1:2:prompt:ChatPromptTemplate] [0ms] Exiting Chain run with output:
{ output: { messages: [...] } }

[llm/start] [1:3:llm:ChatOpenAI] Entering LLM run with messages:
[[ { role: 'human', content: 'You are a helpful assistant...' } ]]

[llm/end] [1:3:llm:ChatOpenAI] [842ms] Exiting LLM run with output:
{ generations: [[{ text: 'Tokyo is the capital of Japan.' }]] }

[chain/end] [1:chain:RunnableSequence] [1205ms] Exiting Chain run with output:
{ output: 'Tokyo is the capital of Japan.' }

=== All events fired. Final result: ===
Tokyo is the capital of Japan.
```

What to notice:

1. **Nesting depth is shown** — The numbers in brackets (e.g., `[1:2:prompt:...]`) indicate nesting. The outer chain started first, then the prompt, then the LLM.
2. **Each step shows timing** — `[842ms]` tells you the LLM took 842 milliseconds.
3. **Events come in start/end pairs** — Every `chain/start` has a corresponding `chain/end`.
4. **The chain fires before the LLM** — `handleChainStart` fires for the outer chain first, then `handleLLMStart` fires when execution reaches the model step.

Common mistakes:

- Writing callbacks: `[ConsoleCallbackHandler]` (the class itself) instead of `callbacks: [new ConsoleCallbackHandler()]` (an instance). You must instantiate the class with `new`.
- Forgetting `dotenv.config()` or calling it after the imports — the API key will not be found.

How to verify success: You should see callback output appearing before the "Final result" line. If you only see the final result with no event output before it, the callback was not attached correctly.

Lab 13.2 — Build a Custom Callback Handler

Learning objective: Write a callback handler from scratch that tracks timing and token usage

across multiple chain invocations.

Why we are doing this: Production applications need custom telemetry — tracking in their own format, writing to their own database, computing their own metrics. This lab teaches the fundamental pattern for any custom handler you will ever write.

Create the file: lab13_2_custom_handler.ts

```
import * as dotenv from "dotenv";
dotenv.config();

import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { BaseCallbackHandler } from "@langchain/core/callbacks/base";
import { LLMResult } from "@langchain/core/outputs";
import { Serialized } from "@langchain/core/load/serializable";

// =====
// CUSTOM HANDLER: TimingAndTokenHandler
// Purpose: Tracks how long each LLM call takes and how many tokens
// are used, both per-call and cumulatively across the session.
// =====
class TimingAndTokenHandler extends BaseCallbackHandler {
  // The `name` property is required and must be unique.
  // It appears in debugging output and LangSmith logs.
  name = "TimingAndTokenHandler";

  // Internal state – these persist across multiple invocations
  // because we create one handler instance and reuse it.
  private llmStartTime: number = 0;
  private chainStartTime: number = 0;
  private cumulativePromptTokens: number = 0;
  private cumulativeCompletionTokens: number = 0;
  private callCount: number = 0;

  // -----
  // Event: handleLLMStart
  // Fires the moment before the LLM API call is made.
  // We record the start time so we can measure duration in handleLLMEnd.
  // -----
  async handleLLMStart(
    llm: Serialized,
    prompts: string[]
  ): Promise<void> {
    this.llmStartTime = Date.now();
    this.callCount++;
    console.log(`\n[HANDLER] === LLM Call #${this.callCount} starting ===`);
    console.log(`[HANDLER] Prompt character count: ${prompts[0]?.length ?? 0}`);
  }

  // -----
  // Event: handleLLMEnd
  // Fires when the LLM finishes generating.
  // The `output` parameter contains the full response and token usage.
  // -----
  async handleLLMEnd(output: LLMResult): Promise<void> {
    const elapsedMs = Date.now() - this.llmStartTime;
    const usage = output.llmOutput?.tokenUsage;

    console.log(`[HANDLER] LLM call #${this.callCount} finished in ${elapsedMs}
ms`);
  }
}
```

```

    if (usage) {
        // Accumulate into session totals
        this.cumulativePromptTokens += usage.promptTokens ?? 0;
        this.cumulativeCompletionTokens += usage.completionTokens ?? 0;

        console.log(`[HANDLER] This call:`);
        console.log(`           Prompt tokens:      ${usage.promptTokens}`);
        console.log(`           Completion tokens: ${usage.completionTokens}`);
        console.log(`           Total tokens:      ${usage.totalTokens}`);
        console.log(`[HANDLER] Session cumulative total: $
{this.getTotalTokensUsed()} tokens`);
    }
}

// -----
// Event: handleChainStart
// Fires when the outer chain (RunnableSequence) begins.
// We record chain start time to measure total pipeline latency.
// -----
async handleChainStart(
    chain: Serialized,
    inputs: Record<string, unknown>
): Promise<void> {
    // chain.id is an array like ["langchain_core", "runnables",
"RunnableSequence"]
    // The last element is the class name.
    const chainName = chain.id?.[chain.id.length - 1] ?? "Unknown";
    this.chainStartTime = Date.now();
    console.log(`[HANDLER] Chain "${chainName}" started`);
}

// -----
// Event: handleChainEnd
// Fires when the chain finishes.
// -----
async handleChainEnd(outputs: Record<string, unknown>): Promise<void> {
    const elapsedMs = Date.now() - this.chainStartTime;
    console.log(`[HANDLER] Chain finished – total pipeline time: ${elapsedMs}
ms`);
}

// -----
// Event: handleLLMError
// Fires on any LLM error. Always implement this in production handlers.
// -----
async handleLLMError(err: Error): Promise<void> {
    console.error(`[HANDLER] LLM ERROR on call #${this.callCount}: $
{err.message}`);
}

// -----
// Public utility methods – called from your application code
// -----
getTotalTokensUsed(): number {
    return this.cumulativePromptTokens + this.cumulativeCompletionTokens;
}

getSessionReport(): object {
    return {
        totalCalls: this.callCount,
        totalPromptTokens: this.cumulativePromptTokens,
        totalCompletionTokens: this.cumulativeCompletionTokens,
        totalTokens: this.getTotalTokensUsed(),
    };
};

```

```

    }
}

// =====
// USING THE CUSTOM HANDLER
// =====
async function main() {
    // Create one handler instance for the entire session.
    // This is important: one instance tracks state across all calls.
    // If you created a new instance per call, the cumulative count would reset.
    const myHandler = new TimingAndTokenHandler();

    const model = new ChatOpenAI({
        modelName: "gpt-3.5-turbo",
        temperature: 0.5,
        callbacks: [myHandler], // attach our custom handler
    });

    const prompt = ChatPromptTemplate.fromTemplate(
        "Summarize this topic in exactly two sentences: {topic}"
    );
    const parser = new StringOutputParser();
    const chain = prompt.pipe(model).pipe(parser);

    // Make three separate calls. The handler accumulates across all of them.
    const topics = ["quantum computing", "machine learning", "blockchain
technology"];

    for (const topic of topics) {
        console.log(`\n${"=".repeat(50)}`);
        console.log(`Querying topic: "${topic}"`);
        const result = await chain.invoke({ topic });
        console.log(`Result: ${result}`);
    }

    // Print the session-wide report after all calls
    console.log(`\n${"=".repeat(50)}`);
    console.log("SESSION REPORT:");
    console.log(JSON.stringify(myHandler.getSessionReport(), null, 2));
}

main().catch(console.error);

```

Run it:

```
npx ts-node lab13_2_custom_handler.ts
```

Expected output (abbreviated):

```

=====
Querying topic: "quantum computing"
[HANDLER] Chain "RunnableSequence" started
[HANDLER] === LLM Call #1 starting ===
[HANDLER] Prompt character count: 74
[HANDLER] LLM call #1 finished in 923ms
[HANDLER] This call:
           Prompt tokens:      24
           Completion tokens:  55
           Total tokens:       79
[HANDLER] Session cumulative total: 79 tokens
[HANDLER] Chain finished – total pipeline time: 1241ms
Result: Quantum computing uses qubits that can exist in multiple states...

```

```
=====
SESSION REPORT:
```

```
{
  "totalCalls": 3,
  "totalPromptTokens": 72,
  "totalCompletionTokens": 163,
  "totalTokens": 235
}
```

Key insight to understand: The handler instance (`myHandler`) persists between the three `chain.invoke()` calls. Its internal variables (`cumulativePromptTokens`, `callCount`, etc.) accumulate across all calls. This is how you build session-level tracking — one handler per session, living as long as the session does.

Lab 13.3 — Set Up LangSmith and Trace a Chain

Learning objective: Configure LangSmith and capture real traces from a multi-step chain. Navigate the LangSmith UI to inspect those traces.

Why we are doing this: Console output is temporary. LangSmith gives you a permanent, searchable, visual record of every run. This is what separates development tooling from production monitoring.

Step 1: Create a LangSmith account

1. Go to smith.langchain.com
2. Sign up for a free account (no credit card required for the free tier)
3. Navigate to Settings → API Keys
4. Click "Create API Key"
5. Give it a name (e.g., "phase5-study")
6. Copy the key (it starts with `ls__`)

Step 2: Create a project in LangSmith

1. In the left sidebar, click "Projects"
2. Click "+ New Project"
3. Name it `langchain-phase5-study`
4. Click Create

Step 3: Update your `.env` file

Add the LangSmith configuration alongside your existing OpenAI key:

```
OPENAI_API_KEY=sk-your-openai-key
LANGCHAIN_TRACING_V2=true
LANGCHAIN_ENDPOINT=https://api.smith.langchain.com
LANGCHAIN_API_KEY=ls__your-langsmith-key-here
LANGCHAIN_PROJECT=langchain-phase5-study
```

Step 4: Create the file: lab13_3_langsmith_trace.ts

```
// CRITICAL: dotenv.config() MUST come before all LangChain imports.
// LangChain reads LANGCHAIN_TRACING_V2 and LANGCHAIN_API_KEY
// at the moment its modules are loaded (when they are imported).
// If you import LangChain first, the env vars are not yet loaded.
import * as dotenv from "dotenv";
dotenv.config();

import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { RunnableSequence } from "@langchain/core/runnables";

// No LangSmith-specific code needed here.
// The env vars handle everything. LangChain automatically
// sends all run data to LangSmith in the background.

async function main() {
  const model = new ChatOpenAI({
    modelName: "gpt-3.5-turbo",
    temperature: 0.7,
  });

  const parser = new StringOutputParser();

  // Chain 1: topic → detailed explanation
  const explainPrompt = ChatPromptTemplate.fromTemplate(
    "Explain {topic} in 3 sentences for a general audience. Be clear and informative."
  );
  const explainChain = explainPrompt.pipe(model).pipe(parser);

  // Chain 2: explanation → simplified version
  const simplifyPrompt = ChatPromptTemplate.fromTemplate(
    "Take this explanation and rewrite it as if explaining to a curious 10-year-old:\n\n{explanation}"
  );

  // Compose the two chains into one pipeline:
  // input.topic → explainChain → explanation → simplifyPrompt → model → parser
  // We use RunnableSequence.from() with an intermediate step that maps
  // the output of explainChain into the input shape that simplifyPrompt
  // expects.
  const fullChain = RunnableSequence.from([
    // Step 1: Run explainChain and rename its output to "explanation"
    {
      explanation: explainChain,
    },
    // Step 2: Feed "explanation" into simplifyPrompt
    simplifyPrompt,
    model,
    parser,
  ]);

  const topics = [
    "how Wi-Fi works",
    "what a database index is",
  ];

  for (const topic of topics) {
    console.log(`\nProcessing topic: "${topic}"...`);
  }
}
```

```

// Pass RunnableConfig with run metadata.
// These show up in LangSmith and help you filter/search traces.
const result = await fullChain.invoke(
  { topic },
  {
    // runName appears as the trace title in LangSmith
    runName: `ExplainAndSimplify-${topic.replace(/\s/g, "-")}`,
    // tags let you filter traces in LangSmith
    tags: ["explain-simplify", "lab-13-3"],
    // metadata is searchable structured data attached to the trace
    metadata: { topic, labNumber: "13.3" },
  }
);

console.log(`Simplified: ${result}`);
}

console.log("\n Done. Open smith.langchain.com to see your traces.");
console.log("   Go to Projects → langchain-phase5-study → Traces");
}

main().catch(console.error);

```

Run it:

```
npx ts-node lab13_3_langsmith_trace.ts
```

What to do in LangSmith after running:

1. Go to smith.langchain.com
2. Click "Projects" → langchain-phase5-study
3. You should see two traces appear (one per topic — it may take a few seconds)
4. Click on ExplainAndSimplify-how-Wi-Fi-works
5. Explore the trace tree:
 - Find the RunnableSequence at the top
 - Expand it to see the two ChatOpenAI nodes (one for each chain)
 - Click on a ChatOpenAI node to see the exact prompt sent and response received
 - Check the token usage on each LLM node

What to look for:

- The trace tree should show two nested ChatOpenAI calls — one for the explanation step, one for the simplification step
- Each LLM node shows prompt tokens, completion tokens, and latency
- The tags explain-simplify and lab-13-3 should be visible on the trace header
- The metadata (topic, labNumber) should appear in the trace details panel

Common mistakes:

- LANGCHAIN_TRACING_V2=True (capital T) does not work — must be lowercase true

- `LANGCHAIN_API_KEY=sk-...` (OpenAI key instead of LangSmith key) — check that you are using the `ls__` key
- Calling `dotenv.config()` after LangChain imports — traces will not appear even though the chain runs fine

How to verify: If you see two traces in LangSmith with nested LLM calls and the correct metadata, the lab is complete.

Lab 13.4 — Track Token Usage Across a RAG Pipeline

Learning objective: Build a RAG pipeline with a custom token tracking callback. Understand how token usage distributes across multiple LLM calls in a single pipeline.

Why we are doing this: RAG pipelines can be expensive because they add retrieved documents to the prompt, inflating token counts. This lab makes that cost visible and measurable.

Install additional dependency:

```
npm install langchain
```

Create the file: `lab13_4_token_tracking_rag.ts`

```
import * as dotenv from "dotenv";
dotenv.config();

import { ChatOpenAI, OpenAIEmbeddings } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { MemoryVectorStore } from "langchain/vectorstores/memory";
import { RunnablePassthrough, RunnableSequence } from
"@langchain/core/runnables";
import { BaseCallbackHandler } from "@langchain/core/callbacks/base";
import { LLMResult } from "@langchain/core/outputs";

// =====
// Token Tracker: records prompt/completion tokens per call
// =====
class TokenTracker extends BaseCallbackHandler {
  name = "TokenTracker";

  // We store a log entry for each LLM call
  private log: Array<{
    callNumber: number;
    promptTokens: number;
    completionTokens: number;
    totalTokens: number;
    timestamp: string;
  }> = [];

  async handleLLMEnd(output: LLMResult): Promise<void> {
    const usage = output.llmOutput?.tokenUsage;
    if (usage) {
      this.log.push({
        callNumber: this.log.length + 1,
        promptTokens: usage.promptTokens ?? 0,
        completionTokens: usage.completionTokens ?? 0,
        totalTokens: usage.totalTokens ?? 0,
        timestamp: new Date().toISOString(),
      });
    }
  }
}
```

```

    });
  }
}

printReport(): void {
  console.log("\n" + "=".repeat(55));
  console.log("TOKEN USAGE REPORT");
  console.log("=".repeat(55));

  let totalPrompt = 0;
  let totalCompletion = 0;

  this.log.forEach((entry) => {
    console.log(`\nCall #${entry.callNumber}:`);
    console.log(`  Prompt tokens:      ${entry.promptTokens}`);
    console.log(`  Completion tokens: ${entry.completionTokens}`);
    console.log(`  Total tokens:        ${entry.totalTokens}`);
    totalPrompt += entry.promptTokens;
    totalCompletion += entry.completionTokens;
  });

  console.log("\n" + "-".repeat(55));
  console.log(`Session total prompt tokens:      ${totalPrompt}`);
  console.log(`Session total completion tokens: ${totalCompletion}`);
  console.log(`Session grand total:                ${totalPrompt +
totalCompletion}`);

  // Rough cost estimate using GPT-3.5-turbo pricing as an example
  const promptCost = (totalPrompt / 1000) * 0.0015;
  const completionCost = (totalCompletion / 1000) * 0.002;
  console.log(`Estimated cost (GPT-3.5-turbo):  $$${(promptCost +
completionCost).toFixed(6)}`);
  console.log("=".repeat(55));
}

// =====
// Knowledge base – our simulated document corpus
// =====
const knowledgeBase = [
  "Node.js is a JavaScript runtime built on Chrome's V8 JavaScript engine that allows JavaScript to run server-side.",
  "TypeScript is a strongly typed programming language that builds on JavaScript by adding optional static type definitions.",
  "Express.js is a minimal and flexible Node.js web application framework that provides routing and middleware features.",
  "PostgreSQL is a powerful, open-source relational database system that supports complex SQL queries and ACID transactions.",
  "REST stands for Representational State Transfer. RESTful APIs use HTTP methods GET, POST, PUT, and DELETE to manage resources.",
  "GraphQL is a query language for APIs that allows clients to request exactly the data they need, avoiding over-fetching.",
  "Docker is a platform for containerising applications, packaging code and dependencies so they run consistently across environments.",
  "A microservices architecture structures an application as a collection of small independent services, each running its own process.",
];

async function main() {
  const tracker = new TokenTracker();

  // Build the vector store from our knowledge base
  console.log("Building vector store (embedding documents)...");
  const embeddings = new OpenAIEmbeddings();

```

```

const vectorStore = await MemoryVectorStore.fromTexts(
  knowledgeBase,
  knowledgeBase.map((_, i) => ({ id: i })),
  embeddings
);
console.log("Vector store ready.\n");

// Build the retriever – returns top 2 most relevant documents
const retriever = vectorStore.asRetriever({ k: 2 });

// The generative model – attach the tracker here
const model = new ChatOpenAI({
  modelName: "gpt-3.5-turbo",
  temperature: 0,
  callbacks: [tracker], // tracker fires on this model's calls
});

// RAG prompt – note the context section that will contain retrieved docs
const ragPrompt = ChatPromptTemplate.fromTemplate(`
You are a helpful technical assistant.
Use ONLY the context below to answer the question.
If the answer is not in the context, say "I don't have that information."

Context:
{context}

Question: {question}

Answer: `);

// Format retrieved documents into a readable string for the prompt
const formatDocs = (docs: Array<{ pageContent: string }>) =>
  docs.map((d, i) => `[${i + 1}] ${d.pageContent}`).join("\n\n");

// Assemble the full RAG chain
const ragChain = RunnableSequence.from([
  {
    // The retriever fetches relevant docs and formatDocs converts them to a
string
    context: retriever.pipe(formatDocs),
    // RunnablePassthrough passes the question through unchanged
    question: new RunnablePassthrough(),
  },
  ragPrompt,
  model,
  new StringOutputParser(),
]);

// Questions to ask – the last one is not in the knowledge base
const questions = [
  "What is Node.js and what does it run on?",
  "What is the difference between REST and GraphQL?",
  "How do I configure a Kubernetes cluster?", // not in knowledge base
];

for (const question of questions) {
  console.log(`Q: ${question}`);
  const answer = await ragChain.invoke(question);
  console.log(`A: ${answer}\n`);
}

// Print the token usage breakdown after all queries
tracker.printReport();
}

```

```
main().catch(console.error);
```

Run it:

```
npx ts-node lab13_4_token_tracking_rag.ts
```

Expected output (approximate):

```
Building vector store (embedding documents)...  
Vector store ready.
```

```
Q: What is Node.js and what does it run on?
```

```
A: Node.js is a JavaScript runtime built on Chrome's V8 JavaScript engine that  
allows JavaScript to run server-side.
```

```
Q: What is the difference between REST and GraphQL?
```

```
A: REST uses HTTP methods like GET, POST, PUT, and DELETE to manage resources,  
while GraphQL is a query language that allows clients to request exactly the  
data they need.
```

```
Q: How do I configure a Kubernetes cluster?
```

```
A: I don't have that information.
```

```
=====
```

TOKEN USAGE REPORT

```
=====
```

Call #1:

```
Prompt tokens:    195  
Completion tokens: 35  
Total tokens:     230
```

Call #2:

```
Prompt tokens:    201  
Completion tokens: 43  
Total tokens:     244
```

Call #3:

```
Prompt tokens:    184  
Completion tokens: 12  
Total tokens:     196
```

```
-----  
Session total prompt tokens:    580  
Session total completion tokens: 90  
Session grand total:           670  
Estimated cost (GPT-3.5-turbo): $0.001047  
=====
```

Key observations:

1. **Prompt tokens dominate** — The retrieved documents add to the prompt, making prompt tokens far outnumber completion tokens. This is typical for RAG.
2. **"I don't have that information" is cheap** — When the model has to say it does not know, completion tokens drop significantly (only 12 in the example above).
3. **Prompt tokens vary slightly** — The two retrieved documents are different for each question, so prompt token counts differ even though the prompt template is the same.

4. **The cost is small at this scale** — But multiply by 10,000 daily users asking 10 questions each, and this becomes significant.
-

Unit 13 — Knowledge Check

Summary

The LangChain callback system provides an event-driven way to observe what your application does internally. You attach handlers (either built-in or custom) to components, and they fire automatically at every step — when LLMs start, when tokens arrive, when chains finish, when tools are used. This gives you visibility into the black box of LLM execution.

LangSmith takes this visibility to the production level: it captures persistent, searchable, visual traces of every run, with zero code changes required (only environment variables). LangSmith traces show the full tree of nested runs, exact inputs and outputs at every step, token usage, latency breakdowns, and support for dataset-based automated evaluation.

Key Takeaways

1. A callback is a function that fires automatically at specific events in the LangChain execution lifecycle.
2. There are events for LLMs, chains, tools, and agents. Every start event has a corresponding end event.
3. Attach callbacks at the constructor level (always fires) or at runtime via `RunnableConfig` (fires only for that call).
4. Write custom handlers by extending `BaseCallbackHandler` and overriding only the events you need.
5. `ConsoleCallbackHandler` is the fastest way to see every event in development.
6. Token usage data is available in `handleLLMEnd` via `output.llmOutput?.tokenUsage`.
7. LangSmith requires only four environment variables — your LangChain code stays unchanged.
8. Always call `dotenv.config()` before importing LangChain modules, or LangSmith tracing will not initialise.
9. LangSmith makes debugging agents dramatically easier by visualising the full ReAct reasoning loop.
10. Token costs are split into prompt tokens and completion tokens — RAG pipelines typically have high prompt token counts because retrieved documents inflate the input.

Practice Questions

1. What is the difference between `handleLLMStart` and `handleChainStart`? Which

- fires first when you invoke a chain that contains an LLM?
2. If you attach a callback handler at the constructor level on a model, and also pass a different handler at `.invoke()` time via `RunnableConfig`, what happens? Do both handlers fire?
 3. What four environment variables does LangSmith require, and what does each one do?
 4. In LangSmith terminology, what is the difference between a "run" and a "trace"?
 5. Why must `dotenv.config()` be called before LangChain imports rather than after?
 6. In a RAG pipeline, which step typically produces the most prompt tokens and why?
 7. What does the `name` property on a `BaseCallbackHandler` subclass do?
 8. If you forget to use `returnExceptions: true` in a batch call and one item fails, what happens to the other items?

Exercises

1. Modify the `TimingAndTokenHandler` from Lab 13.2 to also record wall-clock time between `handleChainStart` and `handleChainEnd`. Print both the LLM time and the total pipeline time.
2. Build a new handler called `JsonFileLogger` that writes every event (name, timestamp, and any relevant data) to a `run_log.json` file on disk using `fs.appendFileSync`.
3. In the LangSmith UI, use the tag filter to find only the traces tagged with `lab-13-3`. How many appear?
4. Modify Lab 13.4 to also print the text of the retrieved documents before each question is answered, by adding a `handleRetrieverEnd` event to the tracker.

Mini Project: Session Cost Estimator

Build a reusable class called `CostEstimator` that:

- Extends `BaseCallbackHandler`
- Accepts prompt and completion price per 1,000 tokens as constructor arguments
- Tracks per-call cost and session-cumulative cost
- Exposes a `getReport()` method returning a structured object with all cost data
- Exposes a `reset()` method that clears accumulated data
- Prints a formatted summary after each chain invocation

Usage target:

```
const estimator = new CostEstimator({
  promptPricePer1000: 0.0015,
  completionPricePer1000: 0.002,
});

const model = new ChatOpenAI({ callbacks: [estimator] });
```

```
// ... run chains ...
console.log(estimator.getReport());
// { calls: 3, totalTokens: 450, estimatedCostUSD: "$0.000810" }
```

UNIT 14: Streaming & Batching

Chapter 4: Streaming — Why Waiting Is Broken UX

4.1 The Problem with the Traditional Approach

Without streaming, calling an LLM works like sending a letter and waiting for a reply:

1. You send the request to the API
2. The LLM generates all tokens internally, one by one
3. Once every token is generated, the complete response is sent back
4. Your application displays it all at once

For a 150-word response generated at roughly 40–50 tokens per second, you might wait 5 to 10 seconds staring at a blank screen, then see all the text appear at once.

This is the worst possible user experience for a conversational interface. Research on perceived performance shows that users rate interfaces as "fast" when they receive feedback within 100ms. An 8-second blank wait feels broken, even if the response is high quality.

Compare this to the experience of using ChatGPT, Claude, or any modern AI chatbot: text appears token by token as it is generated. You can start reading before the response is finished. If the model is going in the wrong direction, you can cancel. Even though the underlying generation time is identical, the experience feels dramatically faster because you get the first token almost immediately.

This is streaming: delivering output incrementally as it is generated, rather than waiting for everything to finish before sending anything.

4.2 How Token-Level Streaming Works

At the API level, streaming works through a mechanism called Server-Sent Events (SSE) or chunked HTTP transfer encoding. The server begins sending data before it has finished generating. The client receives and processes each piece as it arrives.

LangChain abstracts all of this into a simple interface. Every `Runnable` (every chain, model, retriever, or parser) has a `.stream()` method. When you call `.stream()`, it returns an async iterable — a sequence of values that arrive over time, not all at once.

You consume an async iterable with the `for await...of` loop:

```
const stream = await chain.stream({ question: "What is gravity?" });
for await (const chunk of stream) {
```

```
// This executes once for each chunk as it arrives.
// `chunk` is typically a string token or small group of tokens.
process.stdout.write(chunk);
}
```

The `for await...of` loop is JavaScript's mechanism for consuming async iterables. Each iteration waits for the next value to arrive, then processes it, then waits for the next one. This is how you turn a stream of arriving tokens into progressive output.

What is `process.stdout.write`?

You might be used to `console.log()` for printing. But `console.log("hello")` always adds a newline character at the end, printing `hello\n`. When streaming tokens, you want them to flow together on the same line without forced newlines between each token. `process.stdout.write("hello")` prints exactly `hello` with no newline. Use `process.stdout.write` for streaming token display, and `console.log` for standalone lines like headers and completion messages.

4.3 Streaming Propagation Through Chains

Streaming propagates through the entire chain automatically. Consider:

```
ChatPromptTemplate → ChatOpenAI → StringOutputParser
```

When you call `.stream()` on this chain:

- `ChatPromptTemplate` runs synchronously (prompt formatting is instant) and passes the formatted prompt to the next step
- `ChatOpenAI` begins generating and streams tokens as they arrive
- `StringOutputParser` passes each token through as it receives it
- Your code receives each token via `for await...of`

The streaming starts when the LLM begins generating and ends when it finishes. For more complex chains:

```
VectorStoreRetriever → ChatPromptTemplate → ChatOpenAI → StringOutputParser
```

The retriever runs first and completes fully before the LLM starts (retrieval cannot be streamed because you need all the documents before you can construct the prompt). You will notice a pause while retrieval runs, then streaming begins once the LLM starts generating.

4.4 Streaming Events with `.streamEvents()`

Beyond token-level streaming, LangChain provides `.streamEvents()`. While `.stream()` only gives you the final output tokens, `.streamEvents()` gives you an event for every step in the pipeline:

```
const events = await chain.streamEvents(
  { question: "What is TypeScript?" },
  { version: "v2" }
);
```

```

for await (const event of events) {
  // event.event is the event name, like "on_chat_model_stream" or
  "on_retriever_end"
  // event.name is the component name
  // event.data contains the event-specific payload

  if (event.event === "on_retriever_end") {
    const docs = event.data?.output?.documents ?? [];
    console.log(`Retriever found ${docs.length} documents`);
  }

  if (event.event === "on_chat_model_stream") {
    const token = event.data?.chunk?.content ?? "";
    process.stdout.write(token);
  }

  if (event.event === "on_chain_end") {
    console.log("\nChain finished.");
  }
}

```

This is the foundation for building sophisticated streaming UIs:

- Show "Searching knowledge base..." when the retriever fires
- Transition to "Generating answer..." when the LLM starts
- Display a "Done" indicator when the chain ends
- Show a token count as generation progresses

4.5 Streaming in Production APIs

In a real Express.js API, you stream responses using the Server-Sent Events protocol. The client (typically a browser or mobile app) opens a persistent connection and receives tokens as they arrive.

The key HTTP headers for SSE:

```

res.setHeader("Content-Type", "text/event-stream");
res.setHeader("Cache-Control", "no-cache");
res.setHeader("Connection", "keep-alive");
res.setHeader("X-Accel-Buffering", "no"); // disables nginx proxy buffering

```

Each SSE message has the format `data: <payload>\n\n` (two newlines end each message).

The client reads these messages using the browser's `EventSource` API or `fetch` with a readable stream.

```

app.get("/stream", async (req, res) => {
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");

  const stream = await chain.stream({ question: req.query.q });

  for await (const chunk of stream) {
    res.write(`data: ${JSON.stringify({ token: chunk })}\n\n`);
  }

  res.write("data: [DONE]\n\n");
  res.end();
});

```

Chapter 5: Batching — Processing Multiple Inputs Efficiently

5.1 The Sequential Processing Problem

Imagine you need to classify sentiment for 100 customer reviews. Sequential processing:

```
const results = [];  
for (const review of reviews) {  
  const result = await chain.invoke({ text: review }); // ~1 second each  
  results.push(result);  
}  
// Total time: ~100 seconds
```

Each call is a network request to the LLM API. Sequential calls mean you wait for call 1 to finish before starting call 2. With 100 calls at 1 second each, the total time is 100 seconds.

But these are network requests — they can happen concurrently. If you fire 10 simultaneously, all 10 run in parallel and take roughly 1 second total (the time for the slowest of the 10). 100 reviews in groups of 10 would take approximately 10 seconds — a 10x speedup.

Batching in LangChain provides this concurrency automatically, with built-in control over how many calls run simultaneously.

5.2 The `.batch()` Method

```
const inputs = [  
  { text: "This product is amazing!" },  
  { text: "Terrible experience, would not recommend." },  
  { text: "It was okay, nothing special." },  
];  
  
// Process all three in parallel  
const results = await chain.batch(inputs);  
// results[0] → answer for input[0]  
// results[1] → answer for input[1]  
// results[2] → answer for input[2]
```

`.batch()` returns a `Promise<T[]>` — a Promise that resolves when all inputs have been processed. The results array is in the same order as the inputs array, regardless of which calls finished first.

5.3 Controlling Concurrency

LLM APIs have rate limits — a maximum number of requests per minute (RPM) and tokens per minute (TPM). If you fire 100 requests simultaneously, you will likely hit the rate limit and get errors.

Control concurrency with the `maxConcurrency` option:

```
const results = await chain.batch(inputs, {  
  maxConcurrency: 5, // process at most 5 at the same time  
});
```

With `maxConcurrency: 5` and 20 inputs:

- Calls 1–5 start simultaneously
- As each finishes, the next call starts (maintaining the 5-concurrent-limit)
- This continues until all 20 are done

Think of it like a checkout at a supermarket with 5 lanes. New customers fill open lanes immediately when one closes. You always have 5 customers being served at once, never more.

Choosing the right concurrency:

Your API Tier	Recommended maxConcurrency
Free tier	2–3 (strict rate limits)
Pay-as-you-go	5–10
High-volume plan	10–20

Start conservative and increase if you are not hitting rate limit errors.

5.4 Handling Failures in Batch Processing

By default, if any input fails, `.batch()` throws and you lose the results for all inputs — even the ones that succeeded. This is often not what you want.

Use `returnExceptions: true` to capture failures as `Error` objects instead of throwing:

```
const results = await chain.batch(inputs, {
  maxConcurrency: 5,
  returnExceptions: true,
});

// Process results, distinguishing successes from failures
results.forEach((result, index) => {
  if (result instanceof Error) {
    console.error(`Input ${index} failed: ${result.message}`);
    // Log the failure, save to a retry queue, etc.
  } else {
    console.log(`Input ${index} succeeded: ${result}`);
    // Process the successful result
  }
});
```

This pattern is essential in production. If you are processing 1,000 documents and one document causes a timeout error, you want to get results for the other 999, not lose everything.

Chapter 6: Async Methods and `async/await` in Depth

6.1 Why `LangChain` is Fully Asynchronous

Every LLM call is a network request. Network requests take time — milliseconds to seconds. If your application blocked (stopped all execution) while waiting for a network request, it could not serve other users or do anything else during that wait.

Node.js solves this with its event loop. Instead of blocking, Node.js registers a callback for when

the network request finishes, continues doing other work, and then fires the callback when the response arrives. This is asynchronous programming.

Promises and `async/await` are JavaScript's way of working with asynchronous operations in a clean, readable way:

```
// Without async/await (using raw Promises – harder to read)
chain.invoke({ question: "What is AI?" })
  .then(result => console.log(result))
  .catch(err => console.error(err));

// With async/await – reads like synchronous code
async function ask() {
  const result = await chain.invoke({ question: "What is AI?" });
  console.log(result);
}
```

The `await` keyword pauses execution inside the `async` function until the Promise resolves, then continues. Crucially, it does not block Node.js itself — other work continues while waiting.

6.2 Sequential vs Parallel Async Execution

Sequential (slow):

```
// Each await waits for the previous to finish before starting the next
const result1 = await chain.invoke(input1); // waits ~1s
const result2 = await chain.invoke(input2); // waits another ~1s
// Total: ~2s
```

Parallel with `Promise.all` (fast):

```
// Both calls start simultaneously; we wait for both to finish
const [result1, result2] = await Promise.all([
  chain.invoke(input1),
  chain.invoke(input2),
]);
// Total: ~1s (the time for the slower of the two)
```

`Promise.all` takes an array of Promises and returns a Promise that resolves when all of them finish, with an array of their results in the same order.

This is essentially what `.batch()` does internally — it starts all calls in parallel (up to `maxConcurrency`) and collects results in order.

When to use `Promise.all` vs `.batch()`:

- Use `Promise.all` for small, fixed numbers of calls where you control concurrency manually
- Use `.batch()` for variable-length arrays of inputs where you need concurrency control and built-in error handling

6.3 The `ainvoke`, `astream`, `abatch` Methods

In Python, LangChain has an explicit sync/async split because Python has both sync and async function definitions. The async versions have the `a` prefix: `ainvoke`, `astream`, `abatch`.

In TypeScript/JavaScript, all I/O is inherently async — there is no sync version. The `.invoke()`, `.stream()`, and `.batch()` methods in TypeScript are all async (they return Promises). The `a-`prefixed methods exist but are functionally equivalent to their non-prefixed counterparts in most TypeScript contexts.

When you see these in documentation:

- `ainvoke()` — equivalent to `invoke()` in TypeScript
- `astream()` — equivalent to `stream()` in TypeScript
- `abatch()` — equivalent to `batch()` in TypeScript

Use the standard methods (`.invoke()`, `.stream()`, `.batch()`) in TypeScript. The documentation may show `ainvoke` in Python examples, but in TypeScript you use `invoke`.

6.4 Error Handling with `async/await`

Always wrap LLM calls in `try/catch`. LLM APIs can fail for many reasons:

```
try {
  const result = await chain.invoke(input);
  return result;
} catch (error: any) {
  if (error.message?.includes("rate limit")) {
    // Too many requests – back off and retry
    await new Promise(resolve => setTimeout(resolve, 2000));
    return await chain.invoke(input); // retry once
  }
  if (error.message?.includes("context length")) {
    // Input too long for the model
    throw new Error("Input is too long. Please reduce the size.");
  }
  // Unknown error – re-throw
  throw error;
}
```

Chapter 7: RunnableConfig — Runtime Configuration

7.1 What is RunnableConfig?

Every LangChain method that executes something — `.invoke()`, `.stream()`, `.batch()` — accepts an optional second argument called `RunnableConfig`. This is an object that controls how that specific execution behaves.

Think of `RunnableConfig` like the settings you pass to a function call but specifically for LangChain's execution engine. It does not change the chain's logic — it changes how the execution is monitored, traced, and managed.

```
const result = await chain.invoke(
  { question: "What is TypeScript?" }, // the input (first argument)
  {
    // This is RunnableConfig (second argument)
    callbacks: [myHandler],
    tags: ["production", "user-premium"],
```

```

    metadata: { userId: "user-456", sessionId: "sess-xyz" },
    runName: "PremiumUserQuery",
    maxConcurrency: 5,
  }
);

```

7.2 RunnableConfig Fields Explained

callbacks — An array of callback handler instances that should fire for this specific run. These are in addition to any constructor-level callbacks already attached to the chain's components. This is useful for per-request telemetry — for example, a handler that logs to a user-specific database entry.

tags — An array of string labels attached to this run in LangSmith. Tags are how you filter and group traces. Common patterns:

- User tier: "tier:premium", "tier:free"
- Environment: "env:production", "env:staging"
- Feature: "feature:rag-chat", "feature:summarisation"
- Experiment: "experiment:new-prompt-v2"

metadata — A key-value object of structured data attached to this run. Unlike tags (which are simple strings), metadata can contain rich data. This data is searchable in LangSmith. Common patterns:

- User identifier: { userId: "user-456" }
- Request tracing: { requestId: "req-abc-123" }
- Business context: { productId: "prod-789", orderId: "ord-456" }

runName — A display name for this run in LangSmith. Without this, LangSmith shows the class name (e.g., RunnableSequence). With this, you can give runs meaningful names like "SupportChatQuery" or "BatchDocumentSummary".

maxConcurrency — Maximum number of parallel calls for `.batch()`. Only relevant when using `.batch()`, ignored for `.invoke()` and `.stream()`.

recursionLimit — For agents, the maximum number of agent steps before execution stops. Prevents infinite loops. Default is 25.

7.3 Tags and Metadata for Production Tracing

In production, every request should carry identifying metadata:

```

async function handleChatRequest(
  userId: string,
  sessionId: string,
  question: string,
  userTier: "free" | "premium"
): Promise<string> {
  const config = {
    tags: [
      `tier:${userTier}`,
      "endpoint:chat",

```

```

    "env:production",
  ],
  metadata: {
    userId,
    sessionId,
    timestamp: new Date().toISOString(),
    questionLength: question.length,
  },
  runName: `Chat-${userId}-${Date.now()}`,
};

return await chain.invoke({ question }, config);
}

```

In LangSmith, you can then:

- Filter by `tier:premium` to see all premium user traces
- Filter by `endpoint:chat` to see all chat endpoint calls
- Search metadata for a specific `userId` to debug one user's experience
- Compute average latency per tier to see if premium users get faster responses

This is the foundation of production observability for LLM applications.

Chapter 8: Lab Exercises — Streaming & Batching

Lab 14.1 — Stream Chat Model Output Token by Token

Learning objective: Implement progressive token display from an LLM chain. Understand how for `await...of` consumes a stream and why `process.stdout.write` is used.

Why we are doing this: Token-by-token streaming is the single most impactful UX improvement for AI applications. This lab makes the mechanism completely clear.

Create the file: `lab14_1_streaming_tokens.ts`

```

import * as dotenv from "dotenv";
dotenv.config();

import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";

async function main() {
  const model = new ChatOpenAI({
    modelName: "gpt-3.5-turbo",
    temperature: 0.8,
    // Note: you do NOT need to pass `streaming: true` explicitly.
    // When you call `.stream()`, LangChain automatically enables streaming
    // on the underlying API call. Passing `streaming: true` in the constructor
    // is only needed if you want streaming to be the default for `.invoke()`
    too
    // (which streams but collects and returns the full string at the end).
  });

  const prompt = ChatPromptTemplate.fromTemplate(
    "Write a short story (about 100 words) about {subject}. Make it vivid and

```

```

engaging."
);

const parser = new StringOutputParser();
const chain = prompt.pipe(model).pipe(parser);

// === Part 1: Experience the difference – invoke (blocking) ===
console.log("=== Part 1: Standard invoke (blocking) ===");
console.log("Waiting for response...");
const blockingStart = Date.now();
const fullResponse = await chain.invoke({ subject: "a lighthouse keeper" });
const blockingTime = Date.now() - blockingStart;
console.log(fullResponse);
console.log(`\nTime until ANY text appeared: ${blockingTime}ms\n`);

// === Part 2: Streaming ===
console.log("=== Part 2: Streaming (progressive) ===\n");
let firstTokenTime: number | null = null;
const streamStart = Date.now();
let chunkCount = 0;

// .stream() returns an AsyncIterable<string>
// Each chunk is one token or a small group of tokens
const stream = await chain.stream({ subject: "a robot learning to paint" });

for await (const chunk of stream) {
  // Record the time when the first token arrives
  if (firstTokenTime === null) {
    firstTokenTime = Date.now() - streamStart;
  }

  // Write each chunk to stdout WITHOUT a newline.
  // Using console.log here would add a newline after every token,
  // making the output look like a list rather than prose.
  process.stdout.write(chunk);
  chunkCount++;
}

const totalStreamTime = Date.now() - streamStart;

console.log("\n\n=== Streaming statistics ===");
console.log(`Time to FIRST token: ${firstTokenTime}ms`);
console.log(`Total streaming time: ${totalStreamTime}ms`);
console.log(`Total chunks: ${chunkCount}`);
console.log(`Avg chunk rate: ${((chunkCount / (totalStreamTime / 1000)).toFixed(1))} chunks/sec`);

console.log("\n\n=== UX comparison ===");
console.log(`Blocking: user waits ${blockingTime}ms before seeing ANYTHING.`);
console.log(`Streaming: user sees first text after ${firstTokenTime}ms.`);
console.log(`Improvement: ${Math.round(blockingTime / (firstTokenTime ?? 1))}x faster time-to-first-text.`);
}

main().catch(console.error);

```

Run it:

```
npx ts-node lab14_1_streaming_tokens.ts
```

What to observe:

Part 1 will show a delay before anything appears (the full generation time). Part 2 will show text

appearing immediately and building progressively. Look at the "time to first token" statistic — it should be significantly lower than the total generation time.

Experiment: Change the story topic to something that produces a longer response ("write a detailed history of computing"). Notice how streaming becomes even more valuable for longer outputs.

Common mistakes:

- Using `console.log(chunk)` inside the loop — this prints each token on its own line. Always use `process.stdout.write(chunk)` for streaming display.
- Forgetting to call `process.stdout.write("\n")` after the loop — the final cursor position will be on the same line as the last token, so subsequent `console.log()` calls will appear on the same line.

Lab 14.2 — Stream a Full RAG Chain Response

Learning objective: Implement streaming on a complete RAG pipeline. Observe the retrieval pause followed by streaming generation. Understand `streamEvents()` for step-level visibility.

Why we are doing this: RAG chains have a pre-generation phase (retrieval) that cannot stream. Understanding the hybrid nature — blocking retrieval then streaming generation — is essential for building good RAG UIs.

Create the file: `lab14_2_streaming_rag.ts`

```
import * as dotenv from "dotenv";
dotenv.config();

import { ChatOpenAI, OpenAIEmbeddings } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { MemoryVectorStore } from "langchain/vectorstores/memory";
import { RunnablePassthrough, RunnableSequence } from
"@langchain/core/runnables";

// Knowledge base about web development concepts
const knowledgeBase = [
  "TypeScript adds static typing to JavaScript, catching type errors at compile time rather than runtime.",
  "React hooks allow functional components to use state and other React features without writing class components.",
  "REST APIs use stateless HTTP requests where each request contains all information needed to process it.",
  "GraphQL allows clients to specify exactly what data they need, reducing over-fetching compared to REST.",
  "Docker containers package applications and their dependencies, ensuring consistent behaviour across environments.",
  "PostgreSQL supports ACID transactions, ensuring database operations are atomic, consistent, isolated, and durable.",
  "Node.js uses an event-driven, non-blocking I/O model that makes it efficient for I/O-heavy applications.",
  "JWT (JSON Web Token) is a compact way to transmit claims between parties as a JSON object, often used for authentication.",
];

async function buildRAGChain() {
```

```

const embeddings = new OpenAIEmbeddings();
const vectorStore = await MemoryVectorStore.fromTexts(
  knowledgeBase,
  knowledgeBase.map((_, i) => ({ id: i })),
  embeddings
);
const retriever = vectorStore.asRetriever({ k: 2 });

const model = new ChatOpenAI({ modelName: "gpt-3.5-turbo", temperature:
0.3 });

const ragPrompt = ChatPromptTemplate.fromTemplate(`
You are a knowledgeable web development tutor.
Use the context below to answer the question thoroughly and clearly.

Context:
{context}

Question: {question}

Detailed Answer:`);

const formatDocs = (docs: Array<{ pageContent: string }>) =>
  docs.map((d, i) => `[${i + 1}] ${d.pageContent}`).join("\n\n");

return RunnableSequence.from([
  {
    context: retriever.pipe(formatDocs),
    question: new RunnablePassthrough(),
  },
  ragPrompt,
  model,
  new StringOutputParser(),
]);
}

// =====
// Method 1: Basic streaming – just output tokens
// =====
async function streamBasic(chain: any, question: string): Promise<void> {
  console.log(`\nQ: ${question}`);
  console.log("A: ");

  const start = Date.now();
  let firstToken = false;

  const stream = await chain.stream(question);

  for await (const chunk of stream) {
    if (!firstToken) {
      // This pause is the retrieval step running before generation starts
      const retrivalTime = Date.now() - start;
      process.stderr.write(`[retrieval took ~${retrivalTime}ms, now
streaming...]\n`);
      firstToken = true;
    }
    process.stdout.write(chunk);
  }

  console.log("\n" + "-".repeat(60));
}

// =====
// Method 2: streamEvents – step-level visibility

```

```
// =====
async function streamWithEvents(chain: any, question: string): Promise<void> {
  console.log(`\nQ: ${question}`);

  const events = chain.streamEvents(question, { version: "v2" });

  for await (const event of events) {
    // Retriever started – show a progress indicator
    if (event.event === "on_retriever_start") {
      process.stdout.write("[🔍 Searching knowledge base...] ");
    }

    // Retriever finished – show how many docs were found
    if (event.event === "on_retriever_end") {
      const docs = event.data?.output ?? [];
      process.stdout.write(`[Found ${docs.length} documents]\n`);
      process.stdout.write("A: ");
    }

    // LLM streaming token
    if (event.event === "on_chat_model_stream") {
      const token = event.data?.chunk?.content ?? "";
      if (token) {
        process.stdout.write(token);
      }
    }

    // Chain finished
    if (event.event === "on_chain_end" && event.name === "RunnableSequence") {
      process.stdout.write("\n[✅ Generation complete]\n");
    }
  }

  console.log("-".repeat(60));
}

async function main() {
  console.log("Building RAG chain...");
  const ragChain = await buildRAGChain();
  console.log("Ready.\n");

  // === Demo 1: Basic streaming ===
  console.log("=".repeat(60));
  console.log("DEMO 1: Basic Streaming");
  console.log("=".repeat(60));
  await streamBasic(ragChain, "What is the difference between REST and GraphQL?");

  // === Demo 2: Event-level streaming ===
  console.log("\n" + "=".repeat(60));
  console.log("DEMO 2: Event-Level Streaming (streamEvents)");
  console.log("=".repeat(60));
  await streamWithEvents(ragChain, "How does TypeScript improve JavaScript development?");
}

main().catch(console.error);
```

Run it:

```
npx ts-node lab14_2_streaming_rag.ts
```

What to observe in Demo 1:

After you press Enter, nothing happens for a moment (retrieval running). Then you see the [retrieval took ~Xms, now streaming...] message appear, and immediately after, tokens start flowing. The pause between pressing Enter and the first output is retrieval latency. Everything after is generation latency.

What to observe in Demo 2:

You will see [🔍 Searching knowledge base...] appear, then [Found 2 documents], then the answer streaming progressively. This is exactly what you would show in a real chat UI — a "searching..." indicator transitions to the streaming answer.

Key insight: The `streamEvents` approach is the right foundation for any production chat UI. It lets you show different UI states (searching, generating, done) based on which step is running.

Lab 14.3 — Batch 10 Queries and Measure Latency

Learning objective: Process 10 inputs in parallel and compare sequential vs concurrent execution time with real measurements.

Why we are doing this: The performance difference between sequential and batched processing is substantial in real applications. This lab makes the numbers concrete so you understand exactly what batching gives you.

Create the file: `lab14_3_batching_comparison.ts`

```
import * as dotenv from "dotenv";
dotenv.config();

import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";

const model = new ChatOpenAI({
  modelName: "gpt-3.5-turbo",
  temperature: 0.3,
});

const prompt = ChatPromptTemplate.fromTemplate(
  "In exactly ONE sentence (no more), define the following concept: {concept}"
);

const chain = prompt.pipe(model).pipe(new StringOutputParser());

// The 10 concepts to process
const inputs = [
  { concept: "photosynthesis" },
  { concept: "blockchain" },
  { concept: "democracy" },
  { concept: "machine learning" },
  { concept: "compound interest" },
  { concept: "osmosis" },
  { concept: "recursion in programming" },
  { concept: "supply and demand" },
  { concept: "TCP/IP protocol" },
  { concept: "natural selection" },
];
```

```
// =====
// Method 1: Sequential – one at a time
// =====
async function sequential(): Promise<{ results: string[]; elapsedMs: number }> {
  console.log("\n--- Running sequential (one at a time) ---");
  const start = Date.now();
  const results: string[] = [];

  for (let i = 0; i < inputs.length; i++) {
    const result = await chain.invoke(inputs[i]);
    results.push(result);
    process.stdout.write(`  [{i + 1}/${inputs.length}] done\r`);
  }

  const elapsedMs = Date.now() - start;
  console.log(`\n Completed in ${elapsedMs}ms (${(elapsedMs / 1000).toFixed(2)}
s)`);
  return { results, elapsedMs };
}

// =====
// Method 2: Batch with maxConcurrency: 10 (all at once)
// =====
async function batchFull(): Promise<{ results: string[]; elapsedMs: number }> {
  console.log("\n--- Running full parallel batch (maxConcurrency: 10) ---");
  const start = Date.now();

  const results = await chain.batch(inputs, { maxConcurrency: 10 });

  const elapsedMs = Date.now() - start;
  console.log(` Completed in ${elapsedMs}ms (${(elapsedMs / 1000).toFixed(2)}
s)`);
  return { results: results as string[], elapsedMs };
}

// =====
// Method 3: Batch with maxConcurrency: 3 (conservative)
// =====
async function batchControlled(): Promise<{ results: string[]; elapsedMs: number }> {
  console.log("\n--- Running controlled batch (maxConcurrency: 3) ---");
  const start = Date.now();

  const results = await chain.batch(inputs, { maxConcurrency: 3 });

  const elapsedMs = Date.now() - start;
  console.log(` Completed in ${elapsedMs}ms (${(elapsedMs / 1000).toFixed(2)}
s)`);
  return { results: results as string[], elapsedMs };
}

// =====
// Method 4: Batch with returnExceptions (failure handling demo)
// =====
async function batchWithErrorHandling(): Promise<void> {
  console.log("\n--- Demonstrating returnExceptions: true ---");

  // Intentionally include one bad input to trigger an error
  const mixedInputs = [
    { concept: "gravity" },
    { concept: "" }, // empty string – likely to cause a problem
    { concept: "evolution" },
  ];
}
```

```

const results = await chain.batch(mixedInputs, {
  maxConcurrency: 3,
  returnExceptions: true, // failed items become Error objects, not thrown
});

results.forEach((result, i) => {
  if (result instanceof Error) {
    console.log(` Input ${i}: ❌ FAILED - ${result.message.slice(0, 60)}`);
  } else {
    console.log(` Input ${i}: ❏ ${result}`);
  }
});
}

async function main() {
  // Run all methods
  const seq = await sequential();
  const full = await batchFull();
  const controlled = await batchControlled();

  // Print comparison table
  console.log("\n" + "=".repeat(65));
  console.log("PERFORMANCE COMPARISON");
  console.log("=".repeat(65));
  console.log(
    `Sequential (1 at a time):      ${seq.elapsedMs.toString().padStart(6)}ms
(baseline)`
  );
  console.log(
    `Full parallel (10 at once):    ${full.elapsedMs.toString().padStart(6)}ms
(${(seq.elapsedMs / full.elapsedMs).toFixed(1)}x faster)`
  );
  console.log(
    `Controlled (3 at a time):      $
${controlled.elapsedMs.toString().padStart(6)}ms  (${(seq.elapsedMs /
controlled.elapsedMs).toFixed(1)}x faster)`
  );
  console.log("=".repeat(65));

  // Verify results are the same
  console.log("\nVerifying results match across methods:");
  const allMatch = full.results.every(
    (r, i) => typeof r === "string" && typeof seq.results[i] === "string"
  );
  console.log(` Results are all strings: ${allMatch ? "❏ yes" : "❏ no"}`);

  // Sample results
  console.log("\nSample results (from full batch):");
  full.results.slice(0, 3).forEach((r, i) => {
    console.log(` ${inputs[i].concept}: ${r}`);
  });

  // Error handling demo
  await batchWithErrorHandling();
}

main().catch(console.error);

```

Run it:

```
npx ts-node lab14_3_batching_comparison.ts
```

Expected performance results (approximate):

PERFORMANCE COMPARISON

```
=====
Sequential (1 at a time):      9841ms  (baseline)
Full parallel (10 at once):    1823ms  (5.4x faster)
Controlled (3 at a time):      3612ms  (2.7x faster)
=====
```

Key observations:

1. The full parallel batch is roughly as fast as a single call — because all 10 run simultaneously and you wait for the slowest one.
2. Controlled concurrency (3 at a time) is faster than sequential but slower than full parallel. It is the safe choice when rate limits are a concern.
3. The speedup factor varies depending on network latency and API response time. The relative comparison is what matters.

Why results are non-deterministic: LLM generation is inherently variable. Temperatures above 0 introduce randomness. Even at temperature 0, timing varies based on server load and network conditions. The results from sequential and batch calls will not be identical in wording, but they should be semantically equivalent.

Lab 14.4 — Use `async/await` in a Real Node.js HTTP Handler

Learning objective: Build a real Express.js API with three endpoints — single query, streaming, and batch — using proper production patterns including `RunnableConfig`, error handling, and response formatting.

Why we are doing this: Labs so far have been scripts. Real LangChain applications are servers. This lab shows exactly how LangChain fits into an Express.js context with all the real-world concerns: input validation, per-request configuration, streaming HTTP responses, and graceful error handling.

Install Express:

```
npm install express @types/express
```

Create the file: `lab14_4_async_handler.ts`

```
import * as dotenv from "dotenv";
dotenv.config();

import express, { Request, Response, NextFunction } from "express";
import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { RunnableConfig } from "@langchain/core/runnables";

// =====
// IMPORTANT PRODUCTION PATTERN:
// Build the model and chain ONCE at startup, not inside each handler.
//
// Why? Creating a ChatOpenAI or ChatPromptTemplate is cheap, but
// creating them inside every request handler:
//   - Wastes memory (garbage collector has to clean up after each request)
```

```

// - Can cause subtle state issues
// - Is simply unnecessary – chain definitions are stateless and reusable
//
// Create them once here, at module level, before the server starts.
// =====

const model = new ChatOpenAI({
  modelName: "gpt-3.5-turbo",
  temperature: 0.7,
  maxRetries: 2, // retry failed API calls up to 2 times automatically
  timeout: 30_000, // 30 second timeout for each call
});

const prompt = ChatPromptTemplate.fromTemplate(`
You are a helpful assistant for a developer Q&A platform.
Answer the following question clearly and practically. Give code examples where
relevant.

Question: {question}

Answer:`);

const chain = prompt.pipe(model).pipe(new StringOutputParser());

// =====
// EXPRESS APP SETUP
// =====
const app = express();
app.use(express.json());

// Helper: validate and sanitize the question input
function validateQuestion(question: unknown): string {
  if (!question || typeof question !== "string") {
    throw new Error("question is required and must be a string");
  }
  const trimmed = question.trim();
  if (trimmed.length === 0) {
    throw new Error("question cannot be empty");
  }
  if (trimmed.length > 1000) {
    throw new Error("question must be under 1000 characters");
  }
  return trimmed;
}

// Helper: build RunnableConfig from request context
function buildConfig(req: Request, overrides: Partial<RunnableConfig> = {}):
RunnableConfig {
  const userId = (req.headers["x-user-id"] as string) ?? "anonymous";
  const requestId = (req.headers["x-request-id"] as string) ?? `req-${
    Date.now()}`;

  return {
    tags: ["ask-endpoint", `user:${userId}`],
    metadata: {
      userId,
      requestId,
      timestamp: new Date().toISOString(),
      userAgent: req.headers["user-agent"] ?? "unknown",
    },
    runName: `Query-${userId}-${requestId}`,
    ...overrides,
  };
}

```

```
// =====
// ROUTE 1: POST /api/ask
// Standard async invocation – waits for full response before replying
// =====
app.post("/api/ask", async (req: Request, res: Response) => {
  let question: string;

  try {
    question = validateQuestion(req.body.question);
  } catch (err: any) {
    return res.status(400).json({ error: err.message });
  }

  const config = buildConfig(req);

  try {
    const startMs = Date.now();

    // .invoke() waits for the full response before returning.
    // No streaming – the client waits until the complete answer is ready.
    const answer = await chain.invoke({ question }, config);

    const latencyMs = Date.now() - startMs;

    res.json({
      success: true,
      question,
      answer,
      metadata: { latencyMs },
    });
  } catch (error: any) {
    console.error(`[/api/ask] Error:`, error.message);

    // Map LLM errors to appropriate HTTP status codes
    if (error.message?.toLowerCase().includes("rate limit")) {
      return res.status(429).json({
        error: "Rate limit exceeded. Please wait a moment and try again.",
        retryAfterSeconds: 30,
      });
    }
    if (error.message?.toLowerCase().includes("context length")) {
      return res.status(400).json({
        error: "Question plus context is too long for the model. Please shorten your question.",
      });
    }
    if (error.message?.toLowerCase().includes("timeout")) {
      return res.status(504).json({
        error: "The request timed out. Please try again.",
      });
    }

    res.status(500).json({ error: "An internal error occurred." });
  }
});

// =====
// ROUTE 2: GET /api/ask/stream?question=...
// Streaming – sends tokens as Server-Sent Events as they are generated
// =====
app.get("/api/ask/stream", async (req: Request, res: Response) => {
  let question: string;

```

```

try {
  question = validateQuestion(req.query.question);
} catch (err: any) {
  return res.status(400).json({ error: err.message });
}

// Set the headers required for Server-Sent Events.
// These tell the browser to keep the connection open and
// treat incoming data as a stream of events.
res.setHeader("Content-Type", "text/event-stream");
res.setHeader("Cache-Control", "no-cache");
res.setHeader("Connection", "keep-alive");
res.setHeader("X-Accel-Buffering", "no"); // prevent nginx from buffering

// Send an initial "connected" event so the client knows the stream started
res.write("event: connected\ndata: {}\n\n");

const config = buildConfig(req, {
  tags: ["stream-endpoint"],
});

try {
  const stream = await chain.stream({ question }, config);

  for await (const chunk of stream) {
    // Each SSE message must follow the format:
    //   data: <payload>\n\n
    // Two newlines signal the end of one SSE message.
    const payload = JSON.stringify({ token: chunk });
    res.write(`data: ${payload}\n\n`);
  }

  // Signal that streaming is complete
  res.write("event: done\ndata: {}\n\n");
} catch (error: any) {
  console.error(`[/api/ask/stream] Error:`, error.message);
  const errorPayload = JSON.stringify({ message: error.message });
  res.write(`event: error\ndata: ${errorPayload}\n\n`);
} finally {
  res.end();
}
});

// =====
// ROUTE 3: POST /api/batch
// Batch processing – handles multiple questions in one request
// =====
app.post("/api/batch", async (req: Request, res: Response) => {
  const { questions } = req.body;

  // Validate the questions array
  if (!Array.isArray(questions)) {
    return res.status(400).json({ error: "questions must be an array" });
  }
  if (questions.length === 0) {
    return res.status(400).json({ error: "questions array cannot be empty" });
  }
  if (questions.length > 20) {
    return res.status(400).json({
      error: "Maximum 20 questions per batch request",
      received: questions.length,
    });
  }
}

```

```

// Validate each question individually
const validatedInputs: Array<{ question: string }> = [];
const validationErrors: string[] = [];

questions.forEach((q: unknown, i: number) => {
  try {
    validatedInputs.push({ question: validateQuestion(q) });
  } catch (err: any) {
    validationErrors.push(`Question ${i + 1}: ${err.message}`);
  }
});

if (validationErrors.length > 0) {
  return res.status(400).json({ errors: validationErrors });
}

const config = buildConfig(req, {
  tags: ["batch-endpoint"],
  maxConcurrency: 5, // maxConcurrency goes in config for batch
});

try {
  const startMs = Date.now();

  // returnExceptions: true means failed items become Error objects
  // instead of causing the whole batch to throw.
  // This is critical for batch endpoints: one bad question should not
  // invalidate all the other answers.
  const results = await chain.batch(validatedInputs, {
    ...config,
    returnExceptions: true,
  });

  const latencyMs = Date.now() - startMs;

  // Format the response, clearly distinguishing successes from failures
  const formattedResults = results.map((result, i) => ({
    question: questions[i],
    success: !(result instanceof Error),
    answer: result instanceof Error ? null : result as string,
    error: result instanceof Error ? result.message : null,
  }));

  const successCount = formattedResults.filter(r => r.success).length;

  res.json({
    success: true,
    results: formattedResults,
    metadata: {
      total: questions.length,
      succeeded: successCount,
      failed: questions.length - successCount,
      latencyMs,
    },
  });
} catch (error: any) {
  console.error(`[/api/batch] Unexpected error:`, error.message);
  res.status(500).json({ error: "Batch processing failed." });
}
});

// =====
// GLOBAL ERROR HANDLER
// Catches any unhandled errors from route handlers

```

```
// =====
app.use((err: Error, req: Request, res: Response, _next: NextFunction) => {
  console.error("Unhandled error:", err.message);
  res.status(500).json({ error: "Internal server error." });
});

// =====
// START THE SERVER
// =====
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`\n Server running at http://localhost:${PORT}\n`);
  console.log("Available endpoints:");
  console.log("  POST    /api/ask          - Single question (blocking)");
  console.log("  GET     /api/ask/stream    - Single question (streaming SSE)");
  console.log("  POST    /api/batch         - Multiple questions (batch)\n");
});
```

Run the server:

```
npx ts-node lab14_4_async_handler.ts
```

Test all three endpoints with curl:

Test 1 — Single question (blocking):

```
curl -X POST http://localhost:3000/api/ask \
-H "Content-Type: application/json" \
-H "x-user-id: user-alice" \
-d '{"question": "What is the difference between var, let, and const in JavaScript?"}'
```

Expected: a JSON response with the full answer.

Test 2 — Streaming:

```
curl -N "http://localhost:3000/api/ask/stream?
question=Explain+async+await+in+TypeScript"
```

The -N flag disables curl's output buffering so you see tokens as they arrive. You should see SSE data events flowing in one by one.

Test 3 — Batch:

```
curl -X POST http://localhost:3000/api/batch \
-H "Content-Type: application/json" \
-d '{
  "questions": [
    "What is a closure in JavaScript?",
    "What is the event loop in Node.js?",
    "What is TypeScript generics?"
  ]
}'
```

Expected: a JSON response with an array of results, including metadata about how many succeeded.

Test 4 — Error handling (try submitting an empty question):

```
curl -X POST http://localhost:3000/api/ask \
-H "Content-Type: application/json" \
```

```
-d '{"question": ""}'
```

Expected: a 400 error with a clear error message.

Production patterns illustrated in this lab:

1. **Chain built once at startup** — `model`, `prompt`, and `chain` are module-level constants, not recreated per request.
 2. **RunnableConfig per request** — Each request gets tags and metadata with the user ID and request ID, enabling per-user tracing in LangSmith.
 3. **Typed error handling** — Rate limit errors return 429, context length errors return 400, timeouts return 504, and unknown errors return 500. This helps clients respond correctly.
 4. **SSE headers for streaming** — Correct headers prevent proxies from buffering the stream.
 5. **returnExceptions: true on batch** — One failing question does not abort the entire batch.
 6. **Input validation before the LLM call** — Never pass unvalidated user input directly to an LLM chain.
-

Unit 14 — Knowledge Check

Summary

Streaming and batching are the two core performance techniques in production LangChain applications. Streaming eliminates the blank-screen wait by delivering tokens progressively as the LLM generates them, dramatically improving perceived responsiveness. Batching processes multiple inputs concurrently, providing 3–10x throughput improvements over sequential processing with controlled parallelism via `maxConcurrency`.

`RunnableConfig` is the mechanism for runtime configuration — attaching callbacks, tags, metadata, and concurrency settings to any specific invocation. Production applications use `RunnableConfig` to attach user context to every run, enabling per-user tracing and filtering in LangSmith.

Key Takeaways

1. Call `.stream()` instead of `.invoke()` when the user is waiting for a response interactively. Use `for await...of` to consume chunks and `process.stdout.write()` to display them.
2. Use `.streamEvents()` when you need to know which step is running — retrieval, LLM, or parser. This is the foundation for progress indicators in chat UIs.
3. `.batch()` processes multiple inputs concurrently. Always set `maxConcurrency` to avoid rate limit errors.
4. Use `returnExceptions: true` in batch endpoints so one failing input does not abort

all others.

5. `RunnableConfig` can carry tags, metadata, and callbacks to any invocation. Use it to attach user context for LangSmith tracing.
6. Build chains once at module level; do not recreate them per request.
7. Handle LLM errors specifically — rate limits, context length, and timeouts each deserve their own response code and message.
8. Set `X-Accel-Buffering: no` and appropriate SSE headers when streaming from a server behind nginx.

Practice Questions

1. What is the difference between `.stream()` and `.streamEvents()`? When would you choose each?
2. Why use `process.stdout.write(chunk)` instead of `console.log(chunk)` when displaying streaming output?
3. What does `maxConcurrency: 5` mean in a `.batch()` call? What happens if you have 20 inputs?
4. What is `returnExceptions: true` for `.batch()` and why is it important in production endpoints?
5. List five fields that `RunnableConfig` can contain. For each, give a concrete example of when you would use it.
6. Why should chains be built at module level rather than inside request handlers?
7. Which SSE header prevents nginx from buffering the stream?
8. In a RAG pipeline, why does streaming not start immediately when you call `.stream()` — what runs first?

Exercises

1. Modify Lab 14.1 to print the time elapsed between receiving each successive chunk. This gives you the "inter-token latency" — useful for detecting if generation speed changes over a long response.
2. Add a fourth endpoint to Lab 14.4: `GET /api/ask/events` that uses `.streamEvents()` to stream both retriever progress and answer tokens as separate SSE event types.
3. Modify Lab 14.3 to also measure "time to first result" — when does the first completed result come back in each method? (In sequential, the first result comes after one call; in batch, all results come at once after the slowest call finishes.)
4. Build a rate limiter wrapper around `.batch()` that automatically retries failed items (those returned as `Error` objects) after a 2-second delay.

Mini Project: Real-time Content Generator CLI

Build a Node.js CLI tool that:

1. Accepts a topic as a command-line argument: `npx ts-node content_gen.ts "Introduction to Machine Learning"`
 2. Uses streaming to display a generated 5-item outline progressively
 3. Extracts the 5 section titles from the outline
 4. Uses `.batch()` with `maxConcurrency: 3` to generate a 2-sentence summary for each section in parallel
 5. Shows a progress counter: `[3/5 sections complete]` updating as results arrive
 6. Assembles and displays the final document
-

Final Project: Production Document Q&A Service

Overview

You will build a production-ready REST API that combines everything from Phase 5:

- A RAG pipeline serving user questions from indexed documents
- Full LangSmith tracing with per-request metadata
- A custom `CostTracker` callback handler
- Streaming and non-streaming endpoints
- Batch processing for multiple simultaneous questions
- Proper validation, error handling, and observability

Architecture

Client

—	POST /documents	→ Upload & index document text
—	POST /ask	→ Single Q&A (blocking, traced)
—	GET /ask/stream	→ Single Q&A (SSE streaming)
—	POST /batch-ask	→ Multiple questions (batched)
—	GET /stats	→ Token usage & cost report

Requirements

1. Document Indexing (POST /documents)

Accept a JSON body with a `text` field. Split the text into chunks, embed them, and add them to the vector store. Return the number of chunks created.

```
POST /documents
{ "text": "Your long document text goes here..." }
```

```
Response:
{ "success": true, "chunksCreated": 12, "totalDocuments": 12 }
```

2. Single Q&A (POST /ask)

Accept a question. Run the full RAG pipeline. Return the answer with latency and token stats.

```
POST /ask
{
  "question": "What does the document say about X?",
  "userId": "user-123"
}
```

```
Response:
{
  "success": true,
  "question": "...",
  "answer": "...",
  "metadata": {
    "latencyMs": 1842,
    "promptTokens": 412,
    "completionTokens": 87,
    "estimatedCostUSD": "$0.000793"
  }
}
```

3. Streaming Q&A (GET /ask/stream)

Stream the RAG answer as SSE. Send progress events for the retrieval step.

```
event: retrieval_start
data: {}

event: retrieval_end
data: {"documentsFound": 3}

data: {"token": "Based"}
data: {"token": " on"}
data: {"token": " the"}
...

event: done
data: {}
```

4. Batch Q&A (POST /batch-ask)

Accept up to 10 questions. Process with `maxConcurrency: 5`. Return results for all questions.

```
POST /batch-ask
{ "questions": ["What is X?", "Explain Y.", "Define Z."] }

Response:
{
  "results": [
    { "question": "What is X?", "answer": "...", "success": true },
    { "question": "Explain Y.", "answer": "...", "success": true },
    { "question": "Define Z.", "answer": null, "error": "timeout", "success":
false }
  ],
  "metadata": { "total": 3, "succeeded": 2, "failed": 1, "latencyMs": 2341 }
```

```
}
```

5. Stats Endpoint (GET /stats)

Return accumulated token usage and cost for the server's current session.

```
{
  "sessionTokens": {
    "prompt": 3412,
    "completion": 891,
    "total": 4303
  },
  "estimatedCost": {
    "prompt": "$0.005118",
    "completion": "$0.001782",
    "total": "$0.006900"
  },
  "callCount": 17,
  "sessionStarted": "2024-01-15T10:00:00.000Z"
}
```

Key Requirements

Observability:

- All endpoints must have LangSmith tracing enabled via environment variables
- Every request must use `RunnableConfig` with tags (`endpoint:ask`, `endpoint:batch`, etc.) and metadata (`userId`, `requestId`, `timestamp`)
- A `CostTracker` callback handler must accumulate token usage session-wide and power the `/stats` endpoint

Performance:

- Streaming endpoint must use `.streamEvents()` to send retrieval and generation events separately
- Batch endpoint must use `returnExceptions: true`
- Chain and model must be built once at startup

Error handling:

- Rate limit errors → 429 with `retryAfterSeconds`
- Context length errors → 400
- Timeout errors → 504
- Validation errors → 400 with specific field error messages
- All other errors → 500

Code quality:

- TypeScript throughout
- Every function has a clear single responsibility

- No magic numbers (use named constants for concurrency limits, max lengths, etc.)
- Error messages are specific and actionable, not generic

Starter Scaffold

```
import * as dotenv from "dotenv";
dotenv.config();

import express from "express";
import { ChatOpenAI, OpenAIEmbeddings } from "@langchain/openai";
import { MemoryVectorStore } from "langchain/vectorstores/memory";
import { RecursiveCharacterTextSplitter } from "langchain/text_splitter";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { RunnablePassthrough, RunnableSequence } from
"@langchain/core/runnables";
import { BaseCallbackHandler } from "@langchain/core/callbacks/base";
import { LLMResult } from "@langchain/core/outputs";

// --- Constants ---
const MAX_CONCURRENT_BATCH = 5;
const MAX_BATCH_QUESTIONS = 10;
const MAX_QUESTION_LENGTH = 1000;
const CHUNK_SIZE = 500;
const CHUNK_OVERLAP = 50;

// TODO: Implement CostTracker handler
class CostTracker extends BaseCallbackHandler {
  name = "CostTracker";
  // ... your implementation
}

// TODO: Initialize vector store, embeddings, model, and RAG chain at startup

// TODO: Implement all five endpoints

// TODO: Start the server
```

Evaluation Criteria

After completing the project, verify these points:

- ☐ POST /documents successfully indexes documents into the vector store
- ☐ POST /ask returns accurate answers based on indexed documents, with token metadata
- ☐ GET /ask/stream streams tokens progressively with retrieval/generation SSE events
- ☐ POST /batch-ask handles individual failures gracefully (one failure does not break the batch)
- ☐ GET /stats returns accurate cumulative token and cost data
- ☐ LangSmith shows a trace for every request with correct tags and metadata
- ☐ Rate limit errors return 429; timeout errors return 504; validation errors return 400
- ☐ The chain and model are created once at startup, not per request

- ☐ All code is TypeScript with proper types and no `any` where avoidable
-

Appendix: Quick Reference

Callback Handler Template

```
import { BaseCallbackHandler } from "@langchain/core/callbacks/base";
import { LLMResult } from "@langchain/core/outputs";
import { Serialized } from "@langchain/core/load/serializable";

class MyHandler extends BaseCallbackHandler {
  name = "MyHandler";

  async handleLLMStart(llm: Serialized, prompts: string[]): Promise<void> { }
  async handleLLMNewToken(token: string): Promise<void> { }
  async handleLLMEnd(output: LLMResult): Promise<void> { }
  async handleLLMError(err: Error): Promise<void> { }
  async handleChainStart(chain: Serialized, inputs: Record<string, unknown>):
  Promise<void> { }
  async handleChainEnd(outputs: Record<string, unknown>): Promise<void> { }
  async handleChainError(err: Error): Promise<void> { }
  async handleToolStart(tool: Serialized, input: string): Promise<void> { }
  async handleToolEnd(output: string): Promise<void> { }
  async handleToolError(err: Error): Promise<void> { }
}
```

Streaming Quick Reference

```
// Token streaming
const stream = await chain.stream(input);
for await (const chunk of stream) {
  process.stdout.write(chunk);
}
console.log(); // newline after stream

// Event streaming
const events = chain.streamEvents(input, { version: "v2" });
for await (const event of events) {
  switch (event.event) {
    case "on_retriever_end":
      console.log("Retrieval complete");
      break;
    case "on_chat_model_stream":
      process.stdout.write(event.data?.chunk?.content ?? "");
      break;
    case "on_chain_end":
      console.log("\nDone");
      break;
  }
}
```

Batch Quick Reference

```
// Basic batch (all in parallel)
const results = await chain.batch(inputs);

// With concurrency and error handling
```

```
const results = await chain.batch(inputs, {
  maxConcurrency: 5,
  returnExceptions: true,
  tags: ["batch-job"],
  metadata: { jobId: "abc-123" },
});

// Process results
results.forEach((result, i) => {
  if (result instanceof Error) {
    console.error(`Input ${i}: ${result.message}`);
  } else {
    console.log(`Input ${i}: ${result}`);
  }
});
```

RunnableConfig Reference

```
import { RunnableConfig } from "@langchain/core/runnables";

const config: RunnableConfig = {
  callbacks: [myHandler], // handlers for this run only
  tags: ["production", "user:alice"], // LangSmith filter labels
  metadata: { userId: "alice", sessionId: "s1" }, // structured trace data
  runName: "ProductionChatQuery", // display name in LangSmith
  maxConcurrency: 5, // for .batch() only
  recursionLimit: 25, // for agents (default: 25)
};

// Apply to any invocation method
await chain.invoke(input, config);
await chain.stream(input, config);
await chain.batch(inputs, config);
```

LangSmith Environment Variables

```
LANGCHAIN_TRACING_V2=true
LANGCHAIN_ENDPOINT=https://api.smith.langchain.com
LANGCHAIN_API_KEY=ls__your-key-here
LANGCHAIN_PROJECT=your-project-name
```

Place these in your `.env` file and call `dotenv.config()` before any LangChain import.

SSE Server Setup (Express)

```
app.get("/stream", async (req, res) => {
  // Required headers
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");
  res.setHeader("X-Accel-Buffering", "no"); // for nginx

  // Send events
  res.write("event: start\n");

  const stream = await chain.stream(input);
  for await (const chunk of stream) {
    res.write(`data: ${JSON.stringify({ token: chunk })}\n`);
  }
});
```

```
res.write("event: done\ndata: {}\n\n");  
res.end();  
});
```

End of LangChain Mastery Guide — Phase 5: Production

This document covers Unit 13 (Callbacks & Observability) and Unit 14 (Streaming & Batching), with four labs per unit and a final project combining all concepts.